Our first algorithm, insertion sort, solves the ***sorting problem*** introduced in Chapter 1:

Input: A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

Keys: The numbers to be sorted.

Satelite Data: The actual data associated by the keys.

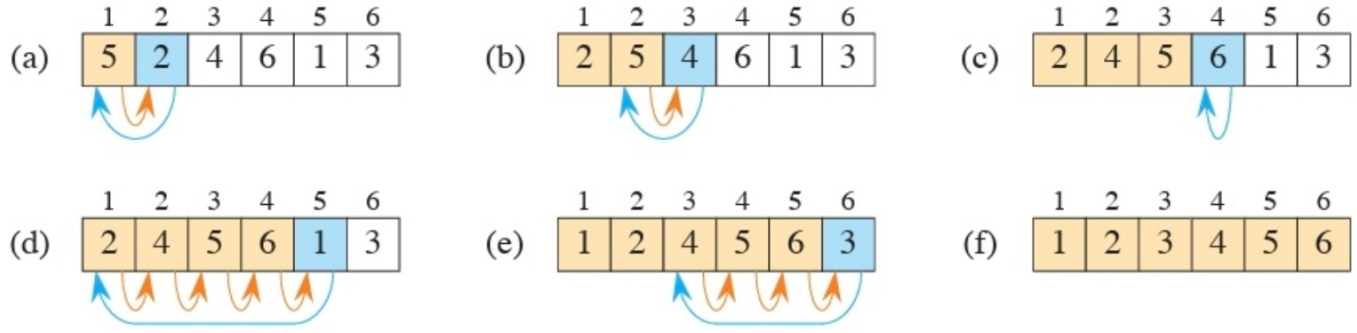
Record: Together with key and satalite data.

Example: Student Record

ID, Name, Email.

Figure 2.1 Sorting a hand of cards using insertion sort.

```
INSERTION-SORT(A, n)
1  for i = 2 to n
2      key = A[i]
3      // Insert A[i] into the sorted subarray A[1 : i − 1].
4      j = i − 1
5      while j > 0 and A[j] > key
6          A[j + 1] = A[j]
7          j = j − 1
8      A[j + 1] = key
```



Correctness of the Algorithm:

Loop Invariance:

Initialization: It is true prior to the first iteration of the loop.

Maintenance: If it is true before an iteration of the loop, it remains true before the next iteration.

Termination: The loop terminates, and when it terminates, the invariant—usually along with the reason that the loop terminated—gives us a useful property that helps show that the algorithm is correct.

Initialization:

$i = 2$, $A[1 : i-1]$ → single key ~~already sorted~~.

Maintenance:

move $A[i]$ on the right one by one until the perfect position $A[i-1] \leq A[i] \leq A[i+1]$

→ Now $A[1 : i]$ sorted.

Increment i.

Termination:

$i \to 2$ to $i = n$.

loop terminates when $i = n + 1$.

Analyzing Algorithm.

Prediction of resource utilization.

Model of Computation:

Assume a one-processor random access machine (RAM) model.

→ Sequential execution i.e. no concurrency.

→ Each instruction / data access takes constant time

→ Data types: integer, float, char.

→ Each word of data has limit on number of bits.

Example: 8 bit char.

Gray Area: Are all available instruction of a computer is allowed in RAM?

Example: Exponentiation operation.

2^n → by left shift is constant time operation.

Avoid Gray Area.

Analysis of Insertion Sort:

Time Requirements:

→ Running machine ?

→ Input size ?

Running Time: Number of instruction and data access executed on a particular input.

INSERTION-SORT(A, n)	*cost*	*times*
1 **for** $i = 2$ **to** n	c_1	n
2 $key = A[i]$	c_2	$n-1$
3 // Insert $A[i]$ into the sorted subarray $A[1:i-1]$.	0	$n-1$
4 $j = i-1$	c_4	$n-1$
5 **while** $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^{n} t_i$
6 $A[j+1] = A[j]$	c_6	$\sum_{i=2}^{n} (t_i - 1)$
7 $j = j-1$	c_7	$\sum_{i=2}^{n} (t_i - 1)$
8 $A[j+1] = key$	c_8	$n-1$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^{n} t_i + c_6 \sum_{i=2}^{n} (t_i - 1) + c_7 \sum_{i=2}^{n} (t_i - 1) + c_8(n-1) .$$

Best case running time:

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= \underbrace{(c_1 + c_2 + c_4 + c_5 + c_8)}_{a} n - \underbrace{(c_2 + c_4 + c_5 + c_8)}_{b} . \end{aligned} \tag{2.1}$$

$$= an - b$$

Worst Case: ?

Input is reverse ordered.

while loop:

$$\sum_{i=2}^{n} t_i = \left(\sum_{i=1}^{n} i\right) - 1 = \frac{n(n+1)}{2} - 1$$

$$\sum_{i=2}^{n} (t_i - 1) = \frac{n(n-1)}{2}.$$

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\ &\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n \\ &\quad - (c_2 + c_4 + c_5 + c_8). \end{aligned}$$

$$= an^2 + bn + c$$

Worst case vs Avg Case:

→ Worst case is the upper bound

→ Some times worst cases happen often

→ Avg case roughly bad as worst.

More Simplification:

Order of growth / Rate of growth:

→ $an^2 + bn + c \approx an^2 \approx n^2$

$\Theta(n^2)$

↳ roughly $\propto n^2$

Will Back To Analysis Soon.

Algorithm Design:

Divide and Conquer:

→ Divide: divide into one or more sub problems of smaller size.

→ Conquer: Solve sub problems recursively

→ Combine: Merge/combine solutions to the original problem.

Merge Sort

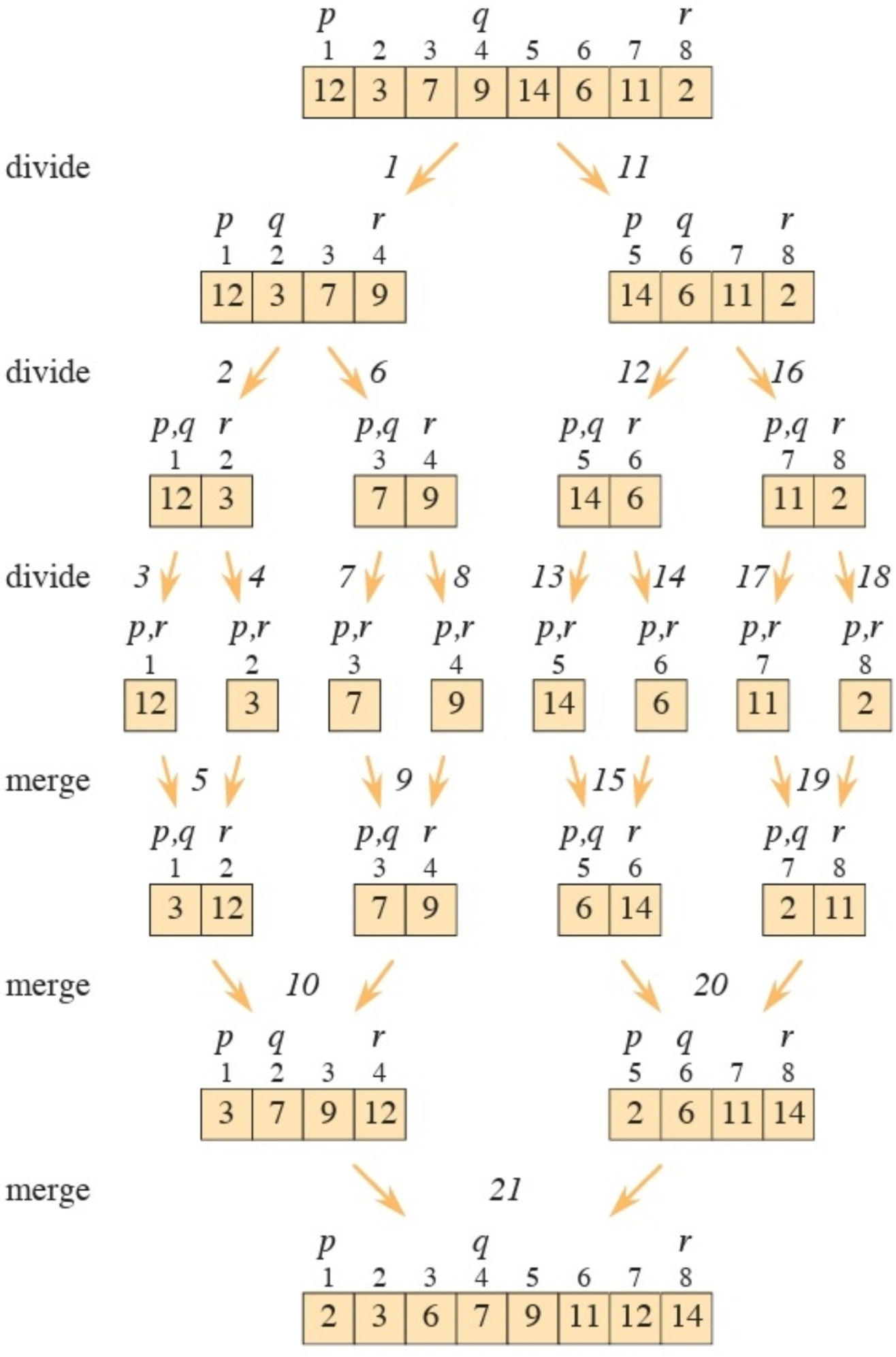
Divide the subarray $A[p:r]$ to be sorted into two adjacent subarrays, each of half the size. To do so, compute the midpoint q of $A[p:r]$ (taking the average of p and r), and divide $A[p:r]$ into subarrays $A[p:q]$ and $A[q+1:r]$.

Conquer by sorting each of the two subarrays $A[p:q]$ and $A[q+1:r]$ recursively using merge sort.

Combine by merging the two sorted subarrays $A[p:q]$ and $A[q+1:r]$ back into $A[p:r]$, producing the sorted answer.

```
MERGE-SORT(A, p, r)
1  if p ≥ r                          // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                   // midpoint of A[p:r]
4  MERGE-SORT(A, p, q)               // recursively sort A[p:q]
5  MERGE-SORT(A, q + 1, r)           // recursively sort A[q+1:r]
6  // Merge A[p:q] and A[q+1:r] into A[p:r].
7  MERGE(A, p, q, r)
```

```
MERGE(A, p, q, r)
 1  n_L = q − p + 1                  // length of A[p:q]
 2  n_R = r − q                      // length of A[q+1:r]
 3  let L[0:n_L − 1] and R[0:n_R − 1] be new arrays
 4  for i = 0 to n_L − 1             // copy A[p:q] into L[0:n_L − 1]
 5      L[i] = A[p + i]
 6  for j = 0 to n_R − 1             // copy A[q+1:r] into R[0:n_R − 1]
 7      R[j] = A[q + j + 1]
 8  i = 0                            // i indexes the smallest remaining element in L
 9  j = 0                            // j indexes the smallest remaining element in R
10  k = p                            // k indexes the location in A to fill
11  // As long as each of the arrays L and R contains an unmerged element,
    //     copy the smallest unmerged element back into A[p:r].
12  while i < n_L and j < n_R
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
18      k = k + 1
19  // Having gone through one of L and R entirely, copy the
    //     remainder of the other to the end of A[p:r].
20  while i < n_L
21      A[k] = L[i]
22      i = i + 1
23      k = k + 1
24  while j < n_R
25      A[k] = R[j]
26      j = j + 1
27      k = k + 1
```


p q r
1 2 3 4 5 6 7 8
12 3 7 9 14 6 11 2
divide
1
11
p q r
1 2 3 4
12 3 7 9
p q r
5 6 7 8
14 6 11 2
divide
2
6
12
16
p,q r
1 2
12 3
p,q r
3 4
7 9
p,q r
5 6
14 6
p,q r
7 8
11 2
divide
3
4
7
8
13
14
17
18
p,r
1
12
p,r
2
3
p,r
3
7
p,r
4
9
p,r
5
14
p,r
6
6
p,r
7
11
p,r
8
2
merge
5
9
15
19
p,q r
1 2
3 12
p,q r
3 4
7 9
p,q r
5 6
6 14
p,q r
7 8
2 11
merge
10
20
p q r
1 2 3 4
3 7 9 12
p q r
5 6 7 8
2 6 11 14
merge
21
p q r
1 2 3 4 5 6 7 8
2 3 6 7 9 11 12 14

$$T(n) = \begin{cases} \Theta(1) & \text{if } n < n_0 , \\ D(n) + aT(n/b) + C(n) & \text{otherwise} . \end{cases}$$

Analysis of merge sort

Here's how to set up the recurrence for $T(n)$, the worst-case running time of merge sort on n numbers.

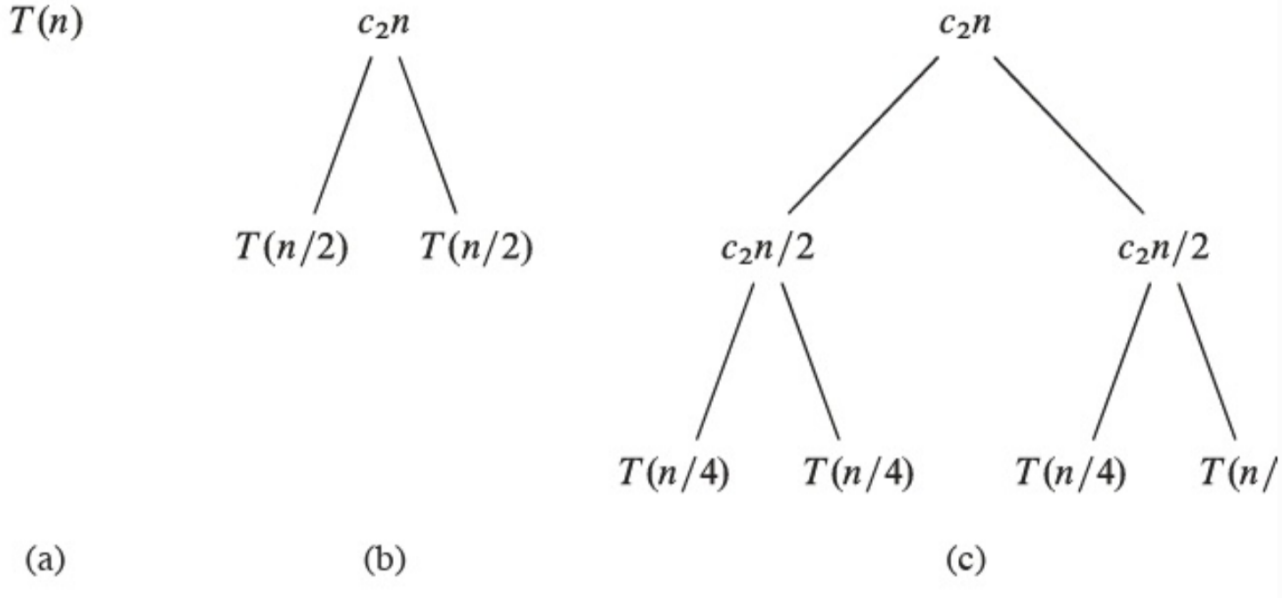
Divide: The divide step just computes the middle of the subarray, which takes constant time. Thus, $D(n) = \Theta(1)$.

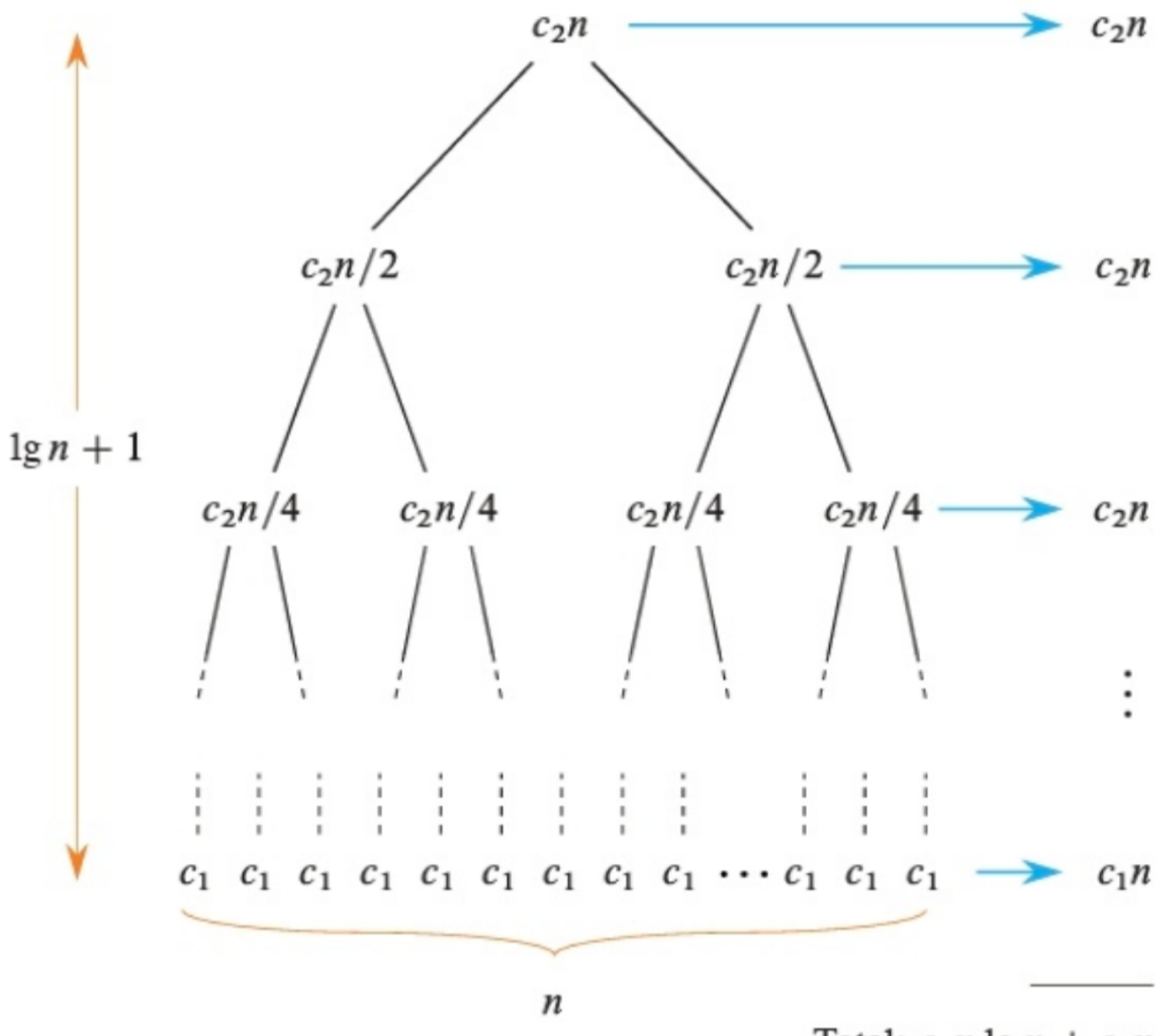
Conquer: Recursively solving two subproblems, each of size $n/2$, contributes $2T(n/2)$ to the running time (ignoring the floors and ceilings, as we discussed).

Combine: Since the MERGE procedure on an n-element subarray takes $\Theta(n)$ time, we have $C(n) = \Theta(n)$.

$$T(n) = 2T(n/2) + \Theta(n)$$

Recurrence Tree:




c_2n
c_2n
$c_2n/2$
$c_2n/2$
c_2n
$\lg n + 1$
$c_2n/4$
$c_2n/4$
$c_2n/4$
$c_2n/4$
c_2n
c_1 c_1 c_1 c_1 c_1 c_1 c_1 c_1 c_1 $\cdots$ c_1 c_1 c_1
c_1n
n
Total: $c_2n \lg n + c_1n$

(d)

Characterizing Running Times

Big O: The upper bound.

$7n^3 + 100n - 20n + 6$ — ①

Growth rate: $n^3 \approx O(n^3)$

also $O(n^4)$, $O(n^6)$

... $O(n^c)$ $c \geq 3$

Big Ω: The lower bound.

For eqn ①: $\Omega(n^3)$

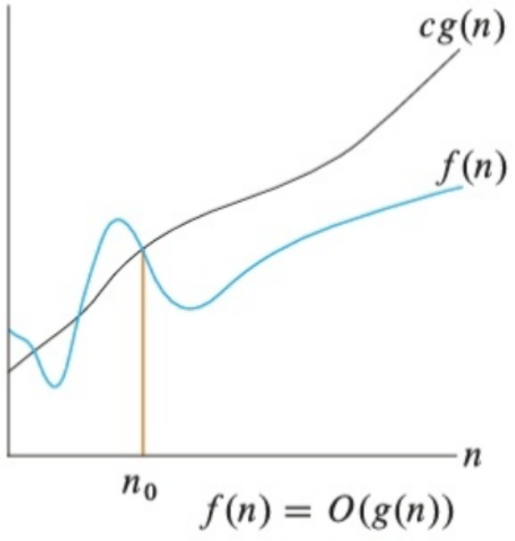
also $\Omega(n^2)$, $\Omega(n)$..

.. $\Omega(n^c)$, $c \leq 3$

Big Θ: The tight bound.

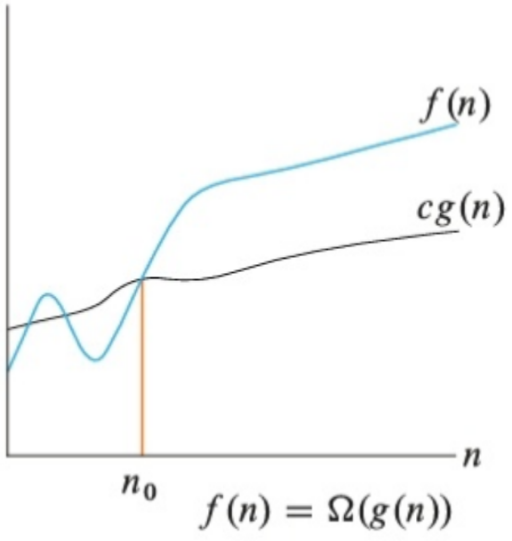
$f(n)$ is both $O(f(n))$ and $\Omega(f(n))$.

For eqn ①: $\Theta(n^3)$.



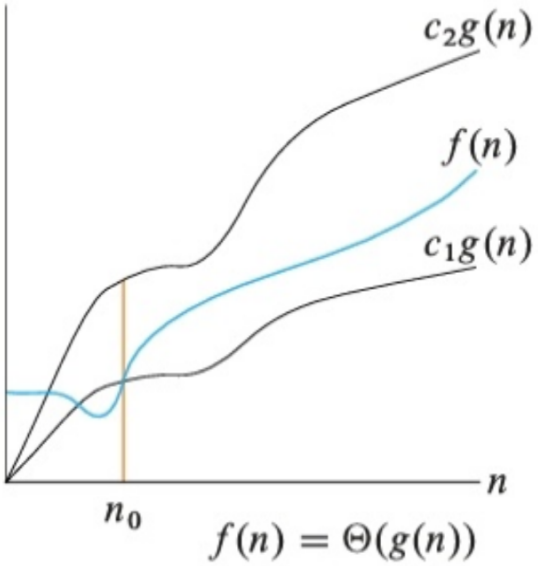
$f(n) = O(g(n))$

(a)



$f(n) = \Omega(g(n))$

(b)



$f(n) = \Theta(g(n))$

(c)

Formal def^{n}:

Big O:

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\}.^{1}$$

Big Ω:

$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0\}.$$

Big Θ:

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0\}.$$

$$f(n) = 4n^2 + 100n + 500$$

For $f(n) = O(n^2)$

$$4n^2 + 100n + 500 \le cn^2 \quad (1)$$

where $c > 0$ and $n > n_0$

$(1) \div n^2$:

$$4 + \frac{100}{n} + \frac{500}{n} \le c$$

there is many c and n_0 for this inequality: like $n_0 = 1 \rightarrow c = 604$

$n_0 = 10 \rightarrow c = 19$

If $f(n) = \Omega(n^2)$

$4n^2 + 100n + 500 \geq cn^2$

where $c > 0$ and $n > n_0$

$\Rightarrow 4 + \frac{100}{n} + \frac{500}{n} \geq c$

true for $c = 4$ and $n_0 > 0$

Now $f(n) = O(n^2) = \Omega(n^2)$

So $f(n) = \Theta(n^2)$

$\therefore f(n) \in \Theta(n^2)$

Little O and Little Ω

Little O: Big O notation may or may not represent asymptotic tight upper bound.

Little O denotes upper bound that is not asymptotic tight

Example: $2n^2 \in O(n^2)$ upper bound

but $2n \notin O(n^2)$ tight

So $2n \in o(n^2)$

Intuition: $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$

then $f(n) = o(g(n))$.

Little ω (omega): Lower bound that is not tight.

Example: $n^2/2 \in \omega(n)$.

Intuition: $\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$

So, $f(n) = \omega(g(n))$.

Solving Recurrence

- **Substitution Method:** Guess the form then use mathematical induction to prove.
- **Recurrence Tree Method:** Model the recurrence tree, each node contains the cost of the levels of recurrence. Add them up
- **Master Method:** Provides bound for the following form of recurrence

$$T(n) = aT(n/b) + f(n)$$

$$a > 0, \quad b > 1.$$

Example:

$$T(n) = 3T(n/4) + \theta(n^2)$$

Substitution Method:

Lets guess $T(n) = \theta(n^2)$

i.e $T(n) \leq dn^2$

where $d > 0, n > n_0$

Inductive hypothesis:

assuming $T(k) \leq dk^2$ for

$k < n$

$$\text{So } T(n/4) \leq d\left(n/4\right)^2$$

$k = n/4 < n$

Substituting the value

$$T(n) \leq 3d\left(n/4\right)^2 + \underbrace{cn^2}_{\theta(n^2)}$$

$$\leq \frac{3dn^2}{16} + cn^2$$

original guess was $T(n) \leq dn^2$

$$\frac{3}{16} dn^2 + cn^2 \leq dn^2$$

$$\frac{3d}{16} + c \leq d$$

$$c \leq d - \frac{3d}{16}$$

$$c \leq \frac{13d}{16}$$

$$\therefore d \geq \frac{16c}{13}$$

now from $\theta(n^2)$ $c > 0$

$$\therefore \frac{16c}{13} > 0$$

$$\therefore d > 0$$

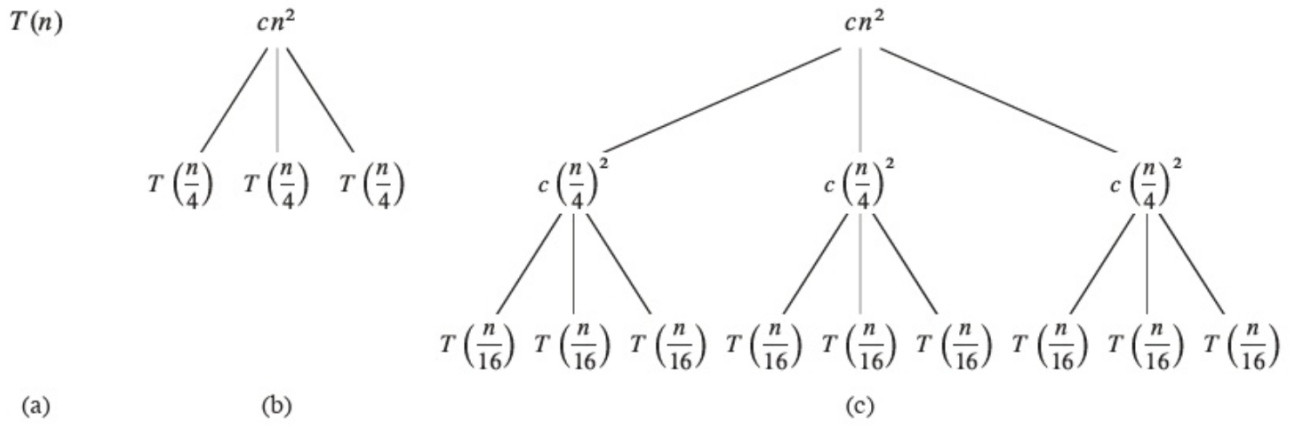
$\therefore T(n) = d\,n^2$ where $d > 0$

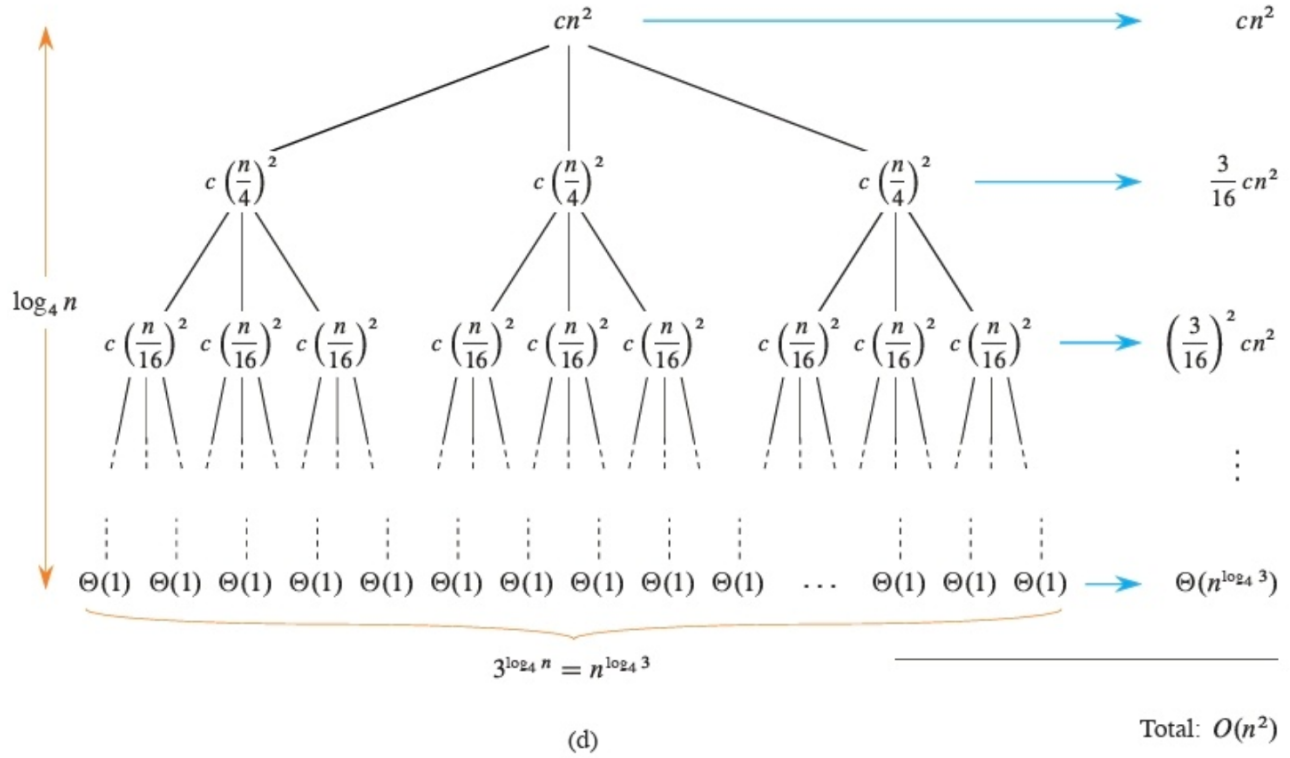
$\therefore T(n) \in \Theta(n) \quad n > n_0$

Base case:

$T(1) = \Theta(1) \leq d(1)^2 = d$

Recurrence Method:





$$
\begin{aligned}
T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n} cn^2 + \Theta(n^{\log_4 3}) \\
&= \sum_{i=0}^{\log_4 n} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{1}{1-(3/16)} cn^2 + \Theta(n^{\log_4 3}) \qquad \text{(by equation (A.7) on page 1142)} \\
&= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\
&= O(n^2) \qquad (\Theta(n^{\log_4 3}) = O(n^{0.8}) = O(n^2)) \,.
\end{aligned}
$$

Master Method:

Theorem 4.1 (Master theorem)
Let $a > 0$ and $b > 1$ be constants, and let $f(n)$ be a driving function that is defined and nonnegative on all sufficiently large reals. Define the recurrence $T(n)$ on $n \in \mathbb{N}$ by

$$T(n) = aT(n/b) + f(n) \,, \tag{4.17}$$

where $aT(n/b)$ actually means $a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil)$ for some constants $a' \geq 0$ and $a'' \geq 0$ satisfying $a = a' + a''$. Then the asymptotic behavior of $T(n)$ can be characterized as follows:

1. If there exists a constant $\epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If there exists a constant $k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$, then $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
3. If there exists a constant $\epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $f(n)$ additionally satisfies the ***regularity condition*** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$. ■

Example:

$$T(n) = 3T(n/4) + \theta(n^2)$$

$$a = 3, \quad b = 4$$

$$f(n) = \theta(n^2) = \Omega(n^2)$$

$$= \Omega(n^{\log_4 16})$$

$$= \Omega(n^{\log_4 3+13})$$

$$\epsilon = 13 > 0$$

$$3f(n/4) \leq cf(n)$$

$$\Rightarrow 3d\left(\frac{n}{4}\right)^2 \leq cdn^2$$

$$\Rightarrow \frac{3dn^2}{16} \leq cdn^2$$

$$\Rightarrow \frac{3}{16} \leq c$$

that c can be $\frac{3}{16}$

which is < 1.

Regularity condition holds

By Masters Method $T(n) = \theta(f(n))$

$= \theta(n^2)$.